

Tarea 2 – Curso De Simulacion

Ejercicios del 1 al 10

Compilado en LaTeX

December 29, 2025

Estudiantes:

- Alexander Brenes Garita
- Owen Torres Porras

Ejercicio 1

Archivo: Ejercicio 1 Tarea 2.py

```
1 #() Ejercicio 1
2 #Utilice el metodo congruencial binario pero en lugar de realizar un xor
3 #con las posiciones 0 y 1, de derecha a izquierda, realice un xor con las
4 #posiciones 0 y 2. Utilice las siguientes semillas y muestre si realiza
5 #recorrido o no.
6
7 #a. x0 = 110
8 #b. x0 = 1111
9
10 #Retorna (mu, lam) donde lam es el periodo (longitud del ciclo)
11 def floyd_cycle(f, x0):
12     tortoise = f(x0)
13     hare = f(f(x0))
14     while tortoise != hare:
15         tortoise = f(tortoise)
16         hare = f(f(hare))
17
18     mu = 0
19     tortoise = x0
20     while tortoise != hare:
21         tortoise = f(tortoise)
22         hare = f(hare)
23         mu += 1
24
25     lam = 1
26     hare = f(tortoise)
27     while tortoise != hare:
28         hare = f(hare)
29         lam += 1
30
31     return mu, lam
32
33 #Un paso del generador binario xor+shift
34 def lfsr_step(state, n_bits, taps=(0, 2)):
35     xor_bit = 0
36     for t in taps:
37         xor_bit ^= (state >> t) & 1
38
39     mask = (1 << n_bits) - 1
40     return ((xor_bit << (n_bits - 1)) | (state >> 1)) & mask
41
42
43 def period_from_seed(seed, n_bits, taps=(0, 2)):
44     f = lambda x: lfsr_step(x, n_bits, taps)
45     mu, lam = floyd_cycle(f, seed)
46     return lam
47
48
49 def fmt_bin(x, n_bits):
```

```

50     return format(x, "0{}b".format(n_bits))
51
52
53 def print_sequence(seed, n_bits, steps=12):
54     x = seed
55     print("Semilla x(0) = {} ({}).format(fmt_bin(seed, n_bits), seed))
56     print("Estados:")
57     for i in range(steps):
58         print("  x({}) = {} -> {}".format(i, fmt_bin(x, n_bits), x))
59         x = lfsr_step(x, n_bits)
60     print("Periodo (desde la semilla): {}".format(period_from_seed(seed,
61         n_bits)))
62
63 def main():
64     print("=== Ejercicio 1: Generador binario (xor taps 0 y 2, shift
65         derecha) ===\n")
66
67     print("a) x(0)=110 (3 bits)")
68     print_sequence(seed=0b110, n_bits=3, steps=10)
69
70     print("\n" + "-" * 60 + "\n")
71
72     print("b) x(0)=1111 (4 bits)")
73     print_sequence(seed=0b1111, n_bits=4, steps=10)
74
75 if __name__ == "__main__":
76     main()

```

Ejercicio 2

Archivo: Ejercicio 2 Tarea 2.py

```
1 #() Ejercicio 2
2 #Para 5 bits construya un metodo congruencial binario que realice un
   recorrido completo. Indique cu l es la semilla y cu l es la
   operaci n que realiza.
3
4 #Primero es necesario mencionar que se va a utilizar las herramientas
   binarias de python como el prefijo 0b para representar binarios y los
   operadores >> , ^ y | para el shift a la derecha, para el xor y para
   el or respectivamente.
5 #Utilizando el metodo congruencial binario solo se puede alcanzar 31
   elementos dentro de los 5 bits ya que el 0 no permite el correcto
   funcionamiento por lo que se colocara una validacion para evitar su
   uso,ademas
6 #utilizando los dos bits menos significativos topamos con el problema de
   que el maximo recorrido se repite cada 21 elementos por lo que
   haciendo un ajuste usamos el primer y tercer bit menos significativo
7 #ya que de esta forma el polinomio generado garantiza un recorrido
   completo.
8
9 #Para esta prueba se usa la semilla 01010 que corresponde a 10 decimal y
   se toma el primer y tercer bit menos significativos usando
   desplazamientos, luego se realizar un xor entre ambos bits para al
   final mezclar por medio de un or
10 #el numero binario actual con el bit mas significativo reemplazado por el
   bit sacado de la operacion del cor
11
12 def congruencialBinario5bits(semilla=0b01010):
13     if semilla == 0:
14         return "La semilla del congruencial binario no puede ser 0"
15
16     recorrido = 32
17     elementos = []
18     actual = semilla & 0b11111
19
20     for i in range(recorrido):
21         elementos.append(actual)
22
23         bit0 = actual & 0b1
24         bit2 = (actual >> 2) & 0b1
25
26         nuevoBit = bit0 ^ bit2
27         actual = (actual >> 1) | (nuevoBit << 4) # Se realiza un or del
           actual con dividido entre 2 y el bit sacado del xor
           multiplicado por 2^4
28
29     return elementos
30
31
32 resultado= congruencialBinario5bits()
33 print(resultado)
```

```
34 if not isinstance(resultado, str):  
35     print(len(resultado))
```

Ejercicio 3

Archivo: Ejercicio 3 Tarea 2.py

```
1 #() Ejercicio 3
2 #Genere 50 numeros aleatorios de 4 d gitos utilizando el generador de
  cuadrados
3 #medios utilizando los siguientes par ametros.
4
5 #a. x0 = 3567345
6 #b. x0 = 1234500012
7 #c. x0 = 4567234902
8
9 def middle_square_next(seed, k=4):
10     sq = seed * seed
11     s = str(sq)
12     if len(s) < k:
13         s = s.zfill(k)
14     start = (len(s) - k) // 2
15     return int(s[start:start + k])
16
17
18 def middle_square_sequence(seed, k=4, n=50):
19     x = seed
20     out = []
21     for _ in range(n):
22         x = middle_square_next(x, k)
23         out.append(x)
24     return out
25
26
27 def print_50(seed):
28     xs = middle_square_sequence(seed, k=4, n=50)
29     print("\nSemilla x(0)={}".format(seed))
30     for i in range(0, 50, 10):
31         chunk = xs[i:i + 10]
32         line = " ".join("{:04d}".format(v) for v in chunk)
33         print("x({:02d})..x({:02d}): {}".format(i + 1, i + len(chunk),
34         line))
35
36
37 def main():
38     print("=== Ejercicio 3: Cuadrados medios (k=4, 50 numeros) ===")
39     for s in [3567345, 1234500012, 4567234902]:
40         print_50(s)
41
42 if __name__ == "__main__":
43     main()
```

Ejercicio 4

Archivo: Ejercicio 4 Tarea 2.py

```
1 #() Ejercicio 4
2 #Determine para cada generador congruencial mixto que se presenta a
   continuaci n si cumple con todo el periodo de m.
3
4 #a. x(0)    = 7
5 #   x(n+1) = ( 5 * x(n) + 24) mod 32
6 #b. x(0)    = 8
7 #   x(n+1) = ( 9 * x(n) + 13) mod 32
8 #c. x(0)    = 13
9 #   x(n+1) = (50 * x(n) + 17) mod 64
10 #d. x(0)    = 15
11 #   x(n+1) = ( 8 * x(n) + 16) mod 100
12 #e. x(0)    = 3
13 #   x(n+1) = ( 5 * x(n) + 21) mod 100
14
15 def congruencialMixto(semilla,multiplicador,constante,modulo):
16     recorrido = []
17     anterior= semilla
18     i=1
19     while i <= modulo+1: #Se agrega uno para ver si el patron se repite
       en caso de que alguno tenga todo el periodo de m
20         recorrido.append(anterior)
21         anterior= (multiplicador * anterior + constante)% modulo
22         i+=1
23     print(recorrido)
24     print("Arreglo de elementos del recorrido sin repetir:
       ",set(recorrido))
25
26
27 #a. x(0)    = 7
28 #   x(n+1) = ( 5 * x(n) + 24) mod 32
29
30 #congruencialMixto(7,5,24,32)
31 # Resultado = [7, 27, 31, 19, 23, 11, 15, 3, 7, 27, 31, 19, 23, 11, 15,
       3, 7, 27, 31, 19, 23, 11, 15, 3, 7, 27, 31, 19, 23, 11, 15, 3, 7]
32 # Arreglo de elementos del recorrido sin repetir: {3, 7, 11, 15, 19, 23,
       27, 31}
33
34 # Al ejecutarlo podemos notar que el patron se repite varias veces y no
       cubre todo el periodo de m.
35
36 #b. x(0)    = 8
37 #   x(n+1) = ( 9 * x(n) + 13) mod 32
38
39 #congruencialMixto(8,9,13,32)
40 # Resultado = [8, 21, 10, 7, 12, 25, 14, 11, 16, 29, 18, 15, 20, 1, 22,
       19, 24, 5, 26, 23, 28, 9, 30, 27, 0, 13, 2, 31, 4, 17, 6, 3, 8]
41 # Arreglo de elementos del recorrido sin repetir: {0, 1, 2, 3, 4, 5, 6,
       7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
       25, 26, 27, 28, 29, 30, 31}
```

```

42
43 # Al ejecutar podemos notar que este generador cumple con todo el periodo
    desde 0 a m - 1
44
45 #c.  $x(0) = 13$ 
46 #  $x(n+1) = (50 * x(n) + 17) \bmod 64$ 
47
48 #congruencialMixto(13,50,17,64)
49 # Resultado = [13, 27, 23, 15, 63, 31, 31, 31, 31, 31, 31, 31, 31, 31,
    31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
    31, 31,
50 # 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31,
    31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31]
51
52 # Arreglo de elementos del recorrido sin repetir: {13, 15, 23, 27, 31,
    63}
53
54 # Al ejecutar podemos notar que este generador no cumple con todo el
    periodo desde 0 a m - 1 ya que en cortas iteraciones el patron se
    bloquea en 31
55
56 #d.  $x(0) = 15$ 
57 #  $x(n+1) = (8 * x(n) + 16) \bmod 100$ 
58
59 #congruencialMixto(15,8,16,100)
60 # Resultado = [15, 36, 4, 48, 0, 16, 44, 68, 60, 96, 84, 88, 20, 76, 24,
    8, 80, 56, 64, 28, 40, 36, 4, 48, 0, 16, 44, 68, 60, 96, 84, 88, 20,
    76, 24, 8, 80, 56, 64, 28, 40, 36, 4,
61 # 48, 0, 16, 44, 68, 60, 96, 84, 88, 20, 76, 24, 8, 80, 56, 64, 28, 40,
    36, 4, 48, 0, 16, 44, 68, 60, 96, 84, 88, 20, 76, 24, 8, 80, 56, 64,
    28, 40, 36, 4, 48, 0, 16, 44, 68, 60, 96, 84, 88, 20, 76, 24, 8, 80,
    56, 64, 28, 40]
62
63 # Arreglo de elementos del recorrido sin repetir: {0, 4, 8, 15, 16, 20,
    24, 28, 36, 40, 44, 48, 56, 60, 64, 68, 76, 80, 84, 88, 96}
64
65 # Podemos observar que este no completa el periodo ya que a partir del 36
    inicia el patron que se repite 5 veces
66
67 #e.  $x(0) = 3$ 
68 #  $x(n+1) = (5 * x(n) + 21) \bmod 100$ 
69
70 #congruencialMixto(3,5,21,100)
71 # Resultado = [3, 36, 1, 26, 51, 76, 1, 26, 51, 76, 1, 26, 51, 76, 1, 26,
    51, 76, 1, 26, 51, 76, 1, 26, 51, 76, 1, 26, 51, 76, 1, 26, 51, 76, 1,
    26, 51, 76, 1, 26, 51, 76, 1, 26, 51, 76, 1, 26, 51,
72 # 76, 1, 26, 51, 76, 1, 26, 51, 76, 1, 26, 51, 76, 1, 26, 51, 76, 1, 26,
    51, 76, 1, 26, 51, 76, 1, 26, 51, 76, 1, 26, 51, 76, 1, 26, 51, 76, 1,
    26, 51, 76, 1, 26, 51, 76, 1, 26, 51]
73
74 # Arreglo de elementos del recorrido sin repetir: {1, 3, 36, 76, 51, 26}
75 # Con los resultados podemos observar que a partir del 1 se genera un
    patron que no permite que se complete todo el periodo

```


Ejercicio 5

Archivo: Ejercicio 5 Tarea 2.py

```
1 #() Ejercicio 5
2 #Determine para cada generador congruencial multiplicativo que se presenta
3 #a continuacion si cumple con todo el periodo de m. La expresion ab
4 #significa a elevado a la potencia b.
5
6 #a. x0 = 7
7 #xn+1 = (5      xn)(26)
8 #b. x0 = 9
9 #xn+1 = (11     xn)(27)
10 #c. x0 = 3
11 #xn+1 = (221    xn)(103)
12 #d. x0 = 17
13 #xn+1 = (203    xn)(105)
14 #e. x0 = 19
15 #xn+1 = (211    xn)(108)
16
17 def floyd_cycle(f, x0):
18     """Retorna (mu, lam) donde lam es el periodo (longitud del ciclo)."""
19     tortoise = f(x0)
20     hare = f(f(x0))
21     while tortoise != hare:
22         tortoise = f(tortoise)
23         hare = f(f(hare))
24
25     mu = 0
26     tortoise = x0
27     while tortoise != hare:
28         tortoise = f(tortoise)
29         hare = f(hare)
30         mu += 1
31
32     lam = 1
33     hare = f(tortoise)
34     while tortoise != hare:
35         hare = f(hare)
36         lam += 1
37
38     return mu, lam
39
40
41 def lcg_mult_step(a, m):
42     return lambda x: (a * x) % m
43
44
45 def main():
46     print("=== Ejercicio 5: Congruencial multiplicativo (periodo con
47         semilla dada) ===\n")
48
49     cases = [
50         ("a", 5, 2**6, 7),
```

```

50     ("b", 11, 2**7, 9),
51     ("c", 221, 10**3, 3),
52     ("d", 203, 10**5, 17),
53     ("e", 211, 10**8, 19),
54 ]
55
56 for label, a, m, x0 in cases:
57     f = lcg_mult_step(a, m)
58     mu, lam = floyd_cycle(f, x0)
59     print("{} ) a={}, m={}, x0={}".format(label, a, m, x0))
60     print("      mu={}, per odo={}".format(mu, lam))
61     print("      Período completo (=m)? {}".format("SI" if lam == m
62     else "NO"))
63     print("")
64
65 if __name__ == "__main__":
66     main()

```

Ejercicio 6

Archivo: Ejercicio 6 Tarea 2.py

```
1 #() Ejercicio 6
2 #Utilice el m todo congruencial mixto y encuentre los par metros
   adecuados de x(0), b, c y m para generar un recorrido completo de
   n meros entre 0 y 14.
3
4 # Es necesario mencionar que se va a usar la funcion del ejercicio 4
   congruencial mixto para calcular el recorrido completo
5 # Vamos a comenzar eligiendo el modulo que seria el modulo 15, necesario
   para que el recorrido completo este entre 0 y 14 ya que m representa
   la secuencia de numeros desde 0 a m-1
6 # Vamos a usar 16 como multiplicador ya que al despejar la congruencia de
   a con 0 de acuerdo al modulo del minimo comun multiplo de los factores
   de m se obtiene 16.
7 # Para la constante vamos a usar c = 1 ya que se recomienda en general.
8 # Para la seleccion de la semilla la unica restriccion es que sea menor
   al modulo por lo que escogeremos x0=1 por lo que tendríamos lo
   siguiente:
9
10 # x(0) = 1
11 # x(n+1) = ( 13 * x(n) + 1) mod 15
12 def congruencialMixto(semilla,multiplicador, constante, modulo):
13     recorrido = []
14     anterior= semilla
15     i=1
16     while i <= modulo+1: #Se agrega uno para ver si el patron se repite
        en caso de que alguno tenga todo el periodo de m
17         recorrido.append(anterior)
18         anterior= (multiplicador * anterior + constante)% modulo
19         i+=1
20     print(recorrido)
21     print("Arreglo de elementos del recorrido sin repetir:
        ",set(recorrido))
22
23
24 congruencialMixto(12,16,1,15)
```

Ejercicio 7

Archivo: Ejercicio 7 Tarea 2.py

```
1 #() Ejercicio 7
2 #Defina los parametros de a, c, x(0) para un recorrido completo con el
  valor
3 #de m = 9.
4
5 def lcg_mixed_step(a, c, m):
6     return lambda x: (a * x + c) % m
7
8
9 def generate(a, c, m, x0, n):
10     f = lcg_mixed_step(a, c, m)
11     x = x0
12     out = [x]
13     for _ in range(n - 1):
14         x = f(x)
15         out.append(x)
16     return out
17
18
19 def main():
20     print("=== Ejercicio 7: Mixto con m=9 (recorrido completo) ===\n")
21
22     a, c, m, x0 = 4, 1, 9, 0
23     seq = generate(a, c, m, x0, n=m)
24
25     print("Par metros: a={}, c={}, m={}, x0={}".format(a, c, m, x0))
26     print("Secuencia (9 valores):")
27     print(seq)
28
29     seen = set(seq)
30     ok = (len(seen) == m)
31     print("\nValores distintos generados:", sorted(seen))
32     print(" Recorre 0..8 completo? {}".format("SI" if ok else "NO"))
33
34
35 if __name__ == "__main__":
36     main()
```

Ejercicio 8

Archivo: Ejercicio 8 Tarea 2.py

```
1 #() Ejercicio 8
2 #Defina los par metros de a, c, x(0) para un recorrido completo con el
  valor de m = 19.
3
4 # Se va a usar la funcion congruencialMixto del ejercicio 4
5 # Ya que tenemos a m = 19 vamos a decir que c = 1 ya que recomiendan el
  numero, la unica restriccion de la semilla es que sea menor a 19 por
  lo que usamos x0= 5
6 # elegimos a = 20 ya que al despejar la congruencia de a con 0 de acuerdo
  al modulo del minimo comun multiplo de los factores de m se obtiene 20.
7 # Con todos los datos tenemos lo siguiente:
8 # x(0) = 5
9 # x(n+1) = ( 20 * x(n) + 1) mod 19
10
11 def congruencialMixto(semilla,multiplicador, constante, modulo):
12     recorrido = []
13     anterior= semilla
14     i=1
15     while i <= modulo+1: #Se agrega uno para ver si el patron se repite
        en caso de que alguno tenga todo el periodo de m
16         recorrido.append(anterior)
17         anterior= (multiplicador * anterior + constante)% modulo
18         i+=1
19     print(recorrido)
20     print("Arreglo de elementos del recorrido sin repetir:
        ",set(recorrido))
21
22 congruencialMixto(5,20,1,19)
```

Ejercicio 9

Archivo: Ejercicio 9 Tarea 2.py

```
1 #() Ejercicio 9
2 #Utilizando las variantes del meetodo congruencial mixto, el metodo
   congruencial
3 #multiplicativo y el metodo congruencial aditivo encuentre generadores
4 #para numeros de 0 hasta 9 que tengan un recorrido completo.
5
6 def lcg_mixed_step(a, c, m):
7     return lambda x: (a * x + c) % m
8
9
10 def lcg_mult_step(a, m):
11     return lambda x: (a * x) % m
12
13
14 def additive_sequence(x0, x1, m, n):
15     out = [x0, x1]
16     a, b = x0, x1
17     for _ in range(n - 2):
18         a, b = b, (a + b) % m
19         out.append(b)
20     return out
21
22
23 def main():
24     print("=== Ejercicio 9: Generadores para 0..9 ===\n")
25
26     # a) mixto (sumar 1 mod 10)
27     print("a) Mixto con per odo 10:  $x(n+1) = (x(n) + 1) \bmod 10$ ")
28     f = lcg_mixed_step(a=1, c=1, m=10)
29     x = 0
30     seq_a = []
31     for _ in range(10):
32         seq_a.append(x)
33         x = f(x)
34     print("    Primeros 10:", seq_a)
35     print("    Distintos:", sorted(set(seq_a)))
36     print("")
37
38     # b) multiplicativo mod 11 y mapeo a 0..9
39     print("b) Multiplicativo mod 11:  $x(n+1)=(2x) \bmod 11$ ,  $x_0=1$ ; luego
        $y=x-1$ ")
40     g = lcg_mult_step(a=2, m=11)
41     x = 1
42     xs = []
43     for _ in range(10):
44         xs.append(x)
45         x = g(x)
46     ys = [v - 1 for v in xs]
47     print("    x (10 valores):", xs)
48     print("    y=x-1:", ys)
```

```

49     print("    y distintos:", sorted(set(ys)))
50     print("")
51
52     # c) aditivo Fibonacci mod 10
53     print("c) Aditivo (Fibonacci) mod 10:  $x(n+1)=(x(n)+x(n-1)) \bmod 10$ ,
          x0=0, x1=1")
54     seq_c = additive_sequence(0, 1, 10, n=20)
55     print("    Primeros 20:", seq_c)
56     print("    Digitos vistos:", sorted(set(seq_c)))
57
58
59 if __name__ == "__main__":
60     main()

```

Ejercicio 10

Archivo: Ejercicio 10 Tarea 2.py

```
1 #() Ejercicio 10
2 #Se tiene un generador aleatorio de un solo dígito. Es decir puede
   generar de manera uniforme los valores 0,1,2,3,4,5,6,7,8 y 9. Para
   cada uno de los siguientes fenómenos indique cómo utilizar a este
   generador.
3
4 #a. Para generar una moneda legal, con 0.50 de probabilidad de que salga
   corona y 0.50 de probabilidad que salga escudo.
5 #b. Para generar el siguiente fenómeno, probabilidad de a es 0.10, la
   probabilidad de b es 0.40 y la probabilidad de c es 0.50.
6 #c. Para generar los valores de arrojar un dado.
7 #d. Si en lugar de un generador aleatorio entre [0,9] se tiene un
   generador de números aleatorios entre [0,99] indique cómo
   realizar la asignación de probabilidades anterior.
8
9 #Para este ejercicio vamos a usar randrange para simular los números
   aleatorios y vamos a suponer que los únicos números que se pueden
   generar son enteros y de un dígito (0,1,2,3,4,5,6,7,8,9)
10
11 from random import randrange
12
13 #a. Para generar una moneda legal, con 0.50 de probabilidad de que salga
   corona y 0.50 de probabilidad que salga escudo.
14
15 # En este problema simplemente se parte a la mitad los números
   generables, del 0 al 4 corresponden a escudos y el resto de 5 a 9
   corresponde a coronas.
16 def moneda(simulaciones):
17     escudos=0
18     coronas=0
19     intento=0
20     for i in range(simulaciones):
21         intento = randrange(0,10,1)
22         if intento < 5:
23             escudos+=1
24             continue
25         coronas+=1
26     print("Escudos: ",escudos/simulaciones)
27     print("Coronas: ",coronas/simulaciones)
28
29 #moneda(1000000)
30
31 #b. Para generar el siguiente fenómeno, probabilidad de a es 0.10, la
   probabilidad de b es 0.40 y la probabilidad de c es 0.50.
32
33 # Este problema se resuelve de una manera similar al anterior solamente
   se divide las áreas de manera distinta.
34 def problemaB(simulaciones):
35     a=0
36     b=0
```



```

37     c=0
38     intento=0
39     for i in range(simulaciones):
40         intento = randrange(0,10,1)
41         if intento == 1:
42             a+=1
43             continue
44         if intento > 1 and intento <= 5:
45             b+=1
46             continue
47         c+=1
48     print("Probabilidad a: ",a/simulaciones)
49     print("Probabilidad b: ",b/simulaciones)
50     print("Probabilidad c: ",c/simulaciones)
51
52 # problemaB(1000000)
53
54 #c. Para generar los valores de arrojar un dado.
55
56 # Para este problema suponiendo que solo se generan numeros enteros de un
57 # solo digito agrupan los valores en los que se van a usar para el
58 # calculo y los que se van a desechar y rehacer
59 # para eso se usan los numeros del 1 al 6 para el calculo y en caso de
60 # que algun numero no pertenezca a ese grupo se rehace el intento
61 def dado(simulaciones):
62     uno=0
63     dos=0
64     tres=0
65     cuatro=0
66     cinco=0
67     seis=0
68     i=0
69     while i != simulaciones:
70         intento = randrange(0,10,1)
71         if intento == 1:
72             uno+=1
73             i+=1
74             continue
75         elif intento == 2:
76             dos+=1
77             i+=1
78             continue
79         elif intento == 3:
80             tres+=1
81             i+=1
82             continue
83         elif intento == 4:
84             cuatro+=1
85             i+=1
86             continue
87         elif intento == 5:
88             cinco+=1
89             i+=1
90             continue

```

```

88         elif intento == 6:
89             seis+=1
90             i+=1
91             continue
92         else:
93             continue
94
95     print("Probabilidad 1: ",uno/simulaciones)
96     print("Probabilidad 2: ",dos/simulaciones)
97     print("Probabilidad 3: ",tres/simulaciones)
98     print("Probabilidad 4: ",cuatro/simulaciones)
99     print("Probabilidad 5: ",cinco/simulaciones)
100    print("Probabilidad 6: ",seis/simulaciones)
101
102    #dado(1000000)
103
104    #d. Si en lugar de un generador aleatorio entre [0,9] se tiene un
105    #generador de numeros aleatorios entre [0,99] indique como realizaria
106    #la asignacion de probabilidades anterior y considere Se pueden
107    #simular dados con todos los poliedros regulares?.
108
109    #Este ejercicio se resuelve como la version normal solo que se adapta los
110    #grupos para abarcar una mayor cantidad
111    def ejercicioAAumentado(simulaciones):
112        aciertos=0
113        fallos=0
114        intento=0
115        for i in range(simulaciones):
116            intento = randrange(0,100,1)
117            if intento < 50:
118                aciertos+=1
119                continue
120            fallos+=1
121        print("Escudos: ",aciertos/simulaciones)
122        print("Coronas: ",fallos/simulaciones)
123
124    #ejercicioAAumentado(1000000)
125
126    #Este ejercicio se resuelve como la version normal solo que se adapta los
127    #grupos para abarcar una mayor cantidad
128    def ejercicioBAumentado(simulaciones):
129        a=0
130        b=0
131        c=0
132        intento=0
133        for i in range(simulaciones):
134            intento = randrange(0,100,1)
135            if intento < 10:
136                a+=1
137                continue
138            if intento > 10 and intento <= 50:
139                b+=1
140                continue
141            c+=1

```

```

137     print("Probabilidad a: ",a/simulaciones)
138     print("Probabilidad b: ",b/simulaciones)
139     print("Probabilidad c: ",c/simulaciones)
140
141 #ejercicioBAumentado(1000000)
142
143 #Para la version adaptada del ejercicio c se puede decir que se pueden
    simular todos los poliedros regulares utilizando la mayor cantidad
    posible de numeros dentro del rango, pero no todos usan el rango
    completo
144
145 # Para el tetraedro se guarda en un arreglo con cuatro espacios que
    representan cada cara, luego se define la cara que se va a incrementar
    de acuerdo al divisor que represente al grupo de la cara, en este caso
146 # 25 ya que 100/4 es 25 por lo que cada cara va a tener 25 elementos
147 # 1/4 = 0.24
148 def tetraedro(simulaciones):
149     caras = [0,0,0,0]
150     i=0
151     while i != simulaciones:
152         intento = randrange(0,100,1)
153         cara = intento // 25
154         caras[cara] += 1
155         i += 1
156
157     for j in range(4):
158         print(f"Probabilidad {j+1}: {caras[j]/simulaciones}")
159
160 #tetraedro(1000000)
161
162 # Para el cubo se guarda en un arreglo con seis espacios que representan
    cada cara, tambien, es necesario conseguir el maximo numero menor a
    100 que sea divisible por 6, en este caso 96, luego se define la cara
    usando el divisor de grupo, en este caso
163 # 16 ya que 96/6 es 16 por lo que cada cara va a tener 16 elementos y si
    es mayor a 96 se rehace el intento
164 # 1/6 = 0.1666666667
165 def cubo(simulaciones):
166     caras = [0,0,0,0,0,0]
167     i = 0
168
169     while i != simulaciones:
170         intento = randrange(0,100,1)
171         if intento < 96:
172             cara = intento // 16
173             caras[cara] += 1
174             i += 1
175
176     for j in range(6):
177         print(f"Probabilidad {j+1}: {caras[j]/simulaciones}")
178
179
180 #cubo(1000000)
181

```

```

182 # Para el octaedro se guarda en un arreglo con ocho espacios que
    representan cada cara, tambien, es necesario conseguir el maximo
    numero menor a 100 que sea divisible por 8, en este caso 88, luego se
    define la cara usando el divisor de grupo, en este caso
183 # 11 ya que 88/8 es 11 por lo que cada cara va a tener 11 elementos y si
    es mayor a 88 se rehace el intento
184 # 1/8 = 0.125
185 def octaedro(simulaciones):
186     caras = [0,0,0,0,0,0,0,0]
187     i = 0
188
189     while i != simulaciones:
190         intento = randrange(0,100,1)
191         if intento < 88:
192             cara = intento // 11
193             caras[cara] += 1
194             i += 1
195
196     for j in range(8):
197         print(f"Probabilidad {j+1}: {caras[j]/simulaciones}")
198 #octaedro(1000000)
199
200 # Para el dodecaedro se guarda en un arreglo con doce espacios que
    representan cada cara, tambien es necesario conseguir el maximo numero
    menor a 100 que sea divisible por 12, en este caso 96, luego se define
    la cara usando el divisor de grupo, en este caso
201 # 8 ya que 96/12 es 8 por lo que cada cara va a tener 8 elementos y si es
    mayor a 96 se rehace el intento
202 # 1/12 = 0.0833333333
203 def dodecaedro(simulaciones):
204     caras = [0,0,0,0,0,0,0,0,0,0,0,0]
205     i = 0
206
207     while i != simulaciones:
208         intento = randrange(0,100,1)
209         if intento < 96:
210             cara = intento // 8
211             caras[cara] += 1
212             i += 1
213
214     for j in range(12):
215         print(f"Probabilidad {j+1}: {caras[j]/simulaciones}")
216
217 #dodecaedro(1000000)
218
219 # Para el icosaedro se guarda en un arreglo con veinte espacios que
    representan cada cara, luego se define la cara usando el divisor de
    grupo, en este caso
220 # 5 ya que 100/20 es 5 por lo que cada cara va a tener 5 elementos.
221 # 1/20 = 0.05
222 def icosaedro(simulaciones):
223     caras = [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
224     i = 0
225

```

```

226     while i != simulaciones:
227         intento = randrange(0,100,1)
228         cara = intento // 5
229         caras[cara] += 1
230         i += 1
231
232     for j in range(20):
233         print(f"Probabilidad {j+1}: {caras[j]/simulaciones}")
234
235 #icosaedro(1000000)

```